

Density-Adaptive Bitmask Encoding: Fixed Chunks with Sparse Lexical Repair

Mainasara Al-amin Tsowa <mainasara@qrk.ng>

Babangida Usman Tsowa <babangida@qrk.ng>

Abstract. Modern subword tokenizers are efficient and robust, but they allocate lexical capacity through a fixed segmentation policy rather than through the local information demand of a span. We study Density-Adaptive Bitmask Encoding (DABE), a learned tokenizer-autoencoder that compresses fixed 64-token chunks into binary codes and spends additional sparse lexical repair capacity only where reconstruction is difficult.

Across TinyStories tokenizer-autoencoder experiments, we find that the most effective design is not adaptive token-window geometry. Instead, a stable fixed chunk paired with a coarse gist stream and a residual-routed sparse lookup stream gives the best rate-distortion tradeoff. The best quality setting reaches 0.91547 token accuracy and 5.41007 mean chunk deviation at 20.37236 observed bits/token. A replicated cost-aware setting identifies `lookup_slot_cost_weight=0.025` as a local compression knee, reaching 0.89274 token accuracy and 6.86454 mean chunk deviation at 20.06207 observed bits/token.

Controlled ablations show that sliding or variable token windows either reduce bitrate at substantial reconstruction cost or collapse to all-fine routing. A matched fixed-rate no-lookup baseline at 20.0 bits/token reaches only 0.72045 token accuracy and 17.89119 mean chunk deviation, compared with the cost-aware DABE point’s 0.89274 token accuracy and 6.86454 mean chunk deviation at 20.06207 observed bits/token. These results identify stable chunks plus adaptive repair as the stronger learned-tokenizer mechanism at matched bitrate.

1. Introduction

Subword tokenizers define the discrete interface between text and modern language models. Before a model sees language, text is committed to a sequence of symbols chosen by a tokenizer, usually a subword segmentation scheme such as byte-pair encoding or a related unigram model [1], [2]. These methods are efficient, robust, and easy to deploy, but their rate allocation is primarily corpus-level: frequent fragments receive compact symbols, rare lexical details are decomposed into longer sequences, and local span difficulty is not represented as an explicit allocation decision.

Natural text has nonuniform information density. A span may contain predictable function words, repeated narrative scaffolding, rare names, punctuation-sensitive dialogue, or local details that are easy to paraphrase but hard to reconstruct exactly. This creates a tokenizer-level rate-distortion question: which parts of a span require lexical precision, and which parts can be represented by a coarse semantic gist?

We study this question through DABE, a learned tokenizer-autoencoder that compresses fixed 64-token chunks into binary codes and reconstructs the original token sequence. The goal is not merely to replace one vocabulary with another. Instead, DABE treats tokenization as a rate-distortion problem: given a finite bit budget, how should a learned tokenizer allocate capacity across a span so that reconstruction error falls where it matters most?

The central design choice in this work is to decouple coarse meaning from lexical repair. A DABE chunk first receives a compact gist representation intended to capture broad local structure. A second, sparse

repair pathway then allocates additional lexical lookup capacity to positions predicted to be difficult. This produces a sliding repair budget over a stable chunk, rather than a sliding token window. The distinction matters: changing the primary token-window geometry can reduce nominal bitrate, but it can also disrupt local code specificity and make reconstruction brittle. In contrast, keeping the chunk stable while adapting the repair budget lets the model preserve a consistent coarse representation and spend precision only where residual error suggests it is useful.

Our experiments on TinyStories establish this framing empirically. The strongest setting, a fixed 64-token chunk with gist encoding and residual-routed sparse lexical lookup, reaches 0.91547 token accuracy and 5.41007 mean chunk deviation at 20.37236 observed bits/token. A cost-aware operating point replicated across adjacent cost settings reaches 0.89274 token accuracy and 6.86454 mean chunk deviation at 20.06207 observed bits/token. A matched fixed-rate no-lookup baseline at 20.0 bits/token is stable and non-collapsed, but reaches only 0.72045 token accuracy and 17.89119 mean chunk deviation. The result is a matched-rate separation: the gain is not explained by total bit budget alone, but by the model’s ability to place lexical precision where reconstruction requires it.

This paper makes four contributions:

- We formulate DABE tokenization as a learned rate-distortion problem over fixed token chunks, with binary codes and explicit reconstruction metrics.
- We introduce a gist-plus-residual-repair architecture that separates coarse span representation from sparse lexical precision.
- We show that adaptive sparse repair substantially outperforms a matched fixed-rate no-lookup learned tokenizer at approximately 20 bits/token.
- We provide controlled ablations showing that adaptive token-window geometry is not, by itself, the right mechanism for this setting.

The paper therefore makes a specific rate-distortion claim: within a controlled tokenizer-autoencoder setting, adaptive sparse lexical repair over stable chunks gives a stronger reconstruction frontier than either fixed-rate learned compression or adaptive token-window geometry. This is the mechanism result that future full language-model integration should build on.

2. Related Work

Subword tokenization. Most contemporary language models use fixed tokenization policies such as byte-pair encoding, WordPiece, unigram language-model tokenization, or byte-level variants [1], [2], [3]. These methods are robust and efficient, but they allocate sequence length through corpus-level frequency statistics rather than through the local information density of a span. DABE keeps the deployment motivation of compact discrete representations, but studies whether a learned tokenizer can allocate reconstruction capacity adaptively inside a fixed chunk.

Learned discrete tokenizers. Discrete autoencoders, vector-quantized models, residual quantizers, and semantic hashing methods learn compact latent codes that can reconstruct or represent text [4], [5]. These approaches motivate treating tokenization as a learned compression problem rather than a fixed segmentation problem. The fixed-rate no-lookup DABE baselines in EXP-071, EXP-079, and EXP-093 play this role in our experiments: they ask how much reconstruction quality a learned chunk code obtains when it spends all capacity uniformly.

Adaptive computation and routing. Adaptive computation mechanisms spend more computation on difficult examples, tokens, or positions [6], [7]. DABE applies the same broad principle to tokenizer reconstruction capacity: the model predicts where sparse lexical repair is worth spending after producing a coarse gist of the chunk. This differs from adaptive windowing because the primary chunk geometry remains stable while the repair budget changes.

Retrieval, copy, and lexical memory. Copy and retrieval mechanisms are often used to recover rare or exact lexical details that parametric sequence models blur [8], [9]. DABE’s sparse lexical lookup stream plays a similar role inside the tokenizer itself: it gives the decoder a small budget for high-precision lexical holes while leaving easier positions to the gist representation.

3. Method

3.1. Problem Setup

Let a tokenized text chunk be

$$x = (x_1, \dots, x_T), \quad x_i \in \mathcal{V}, \quad T = 64. \quad (1)$$

The tokenizer-autoencoder maps this chunk into a binary code and reconstructs a distribution over the original GPT-2 token IDs. We treat the system as a rate-distortion model. A method is useful only when it reduces reconstruction distortion at a comparable observed bitrate.

3.2. Fixed-Rate Chunk Code

The learned chunk encoder produces a binary representation

$$b = q_{\varphi(x)}, \quad b \in \{0, 1\}^B, \quad (2)$$

where B is the base code size. The fixed-rate no-lookup comparator reconstructs directly from this code:

$$h_i = F_{\theta(b,i)}, \quad \ell_i^{\text{base}} = W_o h_i, \quad p_i^{\text{base}} = \text{softmax}(\ell_i^{\text{base}}). \quad (3)$$

The fixed-rate bitrate is therefore

$$R_{\text{fixed}} = \frac{B}{T}. \quad (4)$$

For EXP-093, $B = 1280$ and $T = 64$, giving exactly 20.0 bits/token. This is the matched learned-compression baseline for the cost-aware DABE point.

3.3. Gist Stream

The DABE decoder separates coarse reconstruction from sparse repair. First, a gist decoder maps the binary code into position-wise hidden states and logits:

$$\begin{aligned} g_i &= G_{\theta(b,i)} \\ \ell_i^{\text{gist}} &= W_g g_i \\ p_i^{\text{gist}} &= \text{softmax}(\ell_i^{\text{gist}}). \end{aligned} \quad (5)$$

This stream is responsible for predictable local structure. It should recover easy tokens without spending lexical lookup slots everywhere.

3.4. Residual Difficulty Signal

The residual router estimates where the gist stream is likely to be wrong. In supervised analysis this difficulty can be written as the per-position gist loss

$$d_i = \text{CE}(x_i, p_i^{\text{gist}}) = -\log p_i^{\text{gist}(x_i)}. \quad (6)$$

During decoding, the router uses available hidden features rather than ground-truth labels:

$$r_i = f_{\psi(g_i)}, \quad r_i \in \mathbb{R}^{K_{\text{max}}}, \quad (7)$$

where K_{max} is the maximum number of lexical repair slots that could be exposed for a position or local region. The experiments with residual-router supervision train these scores to correlate with reconstruction difficulty, so high-information holes receive more repair capacity.

3.5. Sparse Lexical Reconstruction

Sparse lexical lookup augments the gist logits with a small set of candidate repair vectors. Let

$$m_{i,k} \in \mathbb{R}^{|\mathcal{V}|}, \quad k = 1, \dots, K_{\max} \quad (8)$$

be the logit-space contribution of lookup slot k at position i . The router converts scores into keep probabilities

$$a_{i,k} = \sigma(r_{i,k}), \quad a_{i,k} \in [0, 1], \quad (9)$$

and the discrete active-slot mask is

$$z_{i,k} = \mathbb{1}[a_{i,k} > \tau]. \quad (10)$$

The repaired reconstruction logits are

$$\ell_i^{\text{repair}} = \ell_i^{\text{gist}} + \lambda_{\text{lookup}} \sum_{k=1}^{K_{\max}} z_{i,k} m_{i,k}, \quad (11)$$

and the final token distribution is

$$p_i^{\text{repair}} = \text{softmax}(\ell_i^{\text{repair}}). \quad (12)$$

This equation is the core sparse-reconstruction claim: the binary chunk code supplies a smooth whole-span gist, while the lookup path fills a small number of lexical holes selected by the residual router.

3.6. Cost-Aware Objective

The reconstruction objective is token cross-entropy over the repaired distribution:

$$\mathcal{L}_{\text{rec}} = \frac{1}{T} \sum_{i=1}^T \text{CE}(x_i, p_i^{\text{repair}}). \quad (13)$$

Cost-aware runs add a penalty for using lookup capacity:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \alpha_{\text{slot}} \frac{1}{TK_{\max}} \sum_{i=1}^T \sum_{k=1}^{K_{\max}} a_{i,k} + \beta_{\text{router}} \mathcal{L}_{\text{router}}. \quad (14)$$

Here α_{slot} is the lookup-slot cost weight and $\mathcal{L}_{\text{router}}$ denotes optional residual-router supervision. The observed effective bitrate charges the base code plus active lexical slots:

$$R_{\text{obs}} = \frac{B + c_{\text{slot}} \sum_{i=1}^T \sum_{k=1}^{K_{\max}} z_{i,k}}{T}. \quad (15)$$

where c_{slot} is the accounting cost per active lookup slot. In the experiments, reported bits/token values use this observed active-slot budget rather than the maximum possible lookup budget. This is why cost-aware points can be compared fairly against fixed-rate no-lookup baselines.

3.7. Distortion Metrics

Token accuracy reports the fraction of positions reconstructed exactly:

$$A_{\text{tok}} = \frac{1}{T} \sum_{i=1}^T \mathbb{1}[\hat{x}_i = x_i], \quad \hat{x}_i = \arg \max_{v \in \mathcal{V}} p_{i(v)}. \quad (16)$$

Chunk deviation is the Hamming distortion over a 64-token slab:

$$D(x, \hat{x}) = \sum_{i=1}^T \mathbb{1}[\hat{x}_i \neq x_i]. \quad (17)$$

We report the mean and p90 of D across validation chunks. Chunk deviation is more informative than exact full-chunk reconstruction in the current regime because even strong models rarely reconstruct all 64 tokens exactly.

3.8. Why Fixed Chunks

The method keeps $T = 64$ fixed and adapts repair allocation through $z_{i,k}$. This is the architectural distinction tested by the paper. A sliding-window tokenizer would make chunk geometry itself adaptive: easy regions could be represented with wider windows, while hard regions could be represented with finer windows. That design is attractive because it appears to align token length with information density. In this setting, however, it also changes the carrier on which the binary code is trained. The decoder must reconstruct not only lexical content, but also a moving segmentation geometry; errors in routing can therefore alter both what is represented and where reconstruction evidence is available.

DABE separates these two problems. The 64-token slab supplies a stable reconstruction coordinate system, while sparse repair changes only the local precision budget. The gist stream can learn a consistent whole-chunk prior, and the residual path can then allocate lexical detail to the holes left by that prior. This is why the model adapts $z_{i,k}$ rather than T : information density changes the repair mask, not the coordinate frame.

We evaluate the sliding-window alternative directly with quantile-supervised variable windows, action-value-supervised window routing, and hard straight-through routing. These variants are reported as controlled ablations: they distinguish whether adaptive granularity should live in token geometry or in repair allocation. The results in Section 5 show that variable windows either reduce bitrate at substantial reconstruction cost or collapse toward all-fine routing, while fixed chunks plus sparse repair maintain the stronger rate-distortion frontier.

4. Experimental Setup

4.1. Dataset and Token Basis

All main tokenizer-autoencoder experiments use TinyStories with 4096 training samples and 512 validation samples in streaming mode. Text is converted to GPT-2 token IDs, and each training example is chunked into fixed 64-token slabs. This keeps the evaluation focused on learned chunk reconstruction rather than on changing the underlying text preprocessing pipeline.

4.2. Training Protocol

The main runs use a Modal T4 GPU, bf16 mixed precision, batch size 32, 12000 maximum training steps, validation every 300 steps, and checkpointing every 2000 train steps. Each experiment logs its configuration, run ID, launch contract, and final summary into `research/experiment_log.md` and `experiments/modal_downloads/`.

4.3. Metrics

We report token accuracy, top-5 and top-10 token accuracy, exact 16-token block accuracy, exact 64-token chunk accuracy, mean chunk deviation, p90 chunk deviation, observed effective bits/token, active lookup K , and bit density. Chunk deviation is the primary distortion metric for comparing reconstructions that are close but not exactly identical.

4.4. Runtime and Compute Accounting

The headline runs are small enough for single-T4 execution but not free: sparse repair adds selector, halting, and lookup-attention work on top of the fixed chunk decoder. Each optimizer step processes $32 \times 64 = 2048$ GPT-2 tokens. The fixed-rate EXP-093 run logged an ETA sample of 14.34 steps/s, or approximately 29368 tokens/s. The cost-aware EXP-092 sweep logged 7.80 steps/s on its first child, or approximately 15974 tokens/s. This indicates roughly a 1.8x training-throughput cost for the sparse repair path relative to the no-lookup matched baseline under the logged T4 runs.

Setting	Base bits	Slot bits	Budget K	Active K	bits/token
EXP-093 fixed-rate no lookup	1280	0	0.00	0.00	20.000
EXP-092 cost-aware sparse repair	1024	22	11.82	6.64	20.062
EXP-087 quality anchor	1024	22	12.72	9.03	20.372

Table 1: Bitrate and repair-budget accounting for the main settings. Budget K determines the charged side budget; active K is the threshold-active lookup usage.

EXP-094 adds a measured decode-profile pass on the EXP-087 checkpoint: 1351 validation chunks were processed in 6.42 seconds, or 210.33 chunks/s and 13461 tokens/s, with CUDA peak allocated/reserved memory of 2031.74/2152.00 MiB. Computationally, the lookup path adds memory and attention activations proportional to $O(B_{\text{batch}} K_{\text{max}} d + B_{\text{batch}} T K_{\text{max}})$ for hidden width d , chunk length $T = 64$, and maximum lookup list $K_{\text{max}} = 32$.

4.5. Baselines and Ablations

The main fixed-rate comparator is EXP-093, a no-lookup hierarchical-local decoder at 20.0 bits/token. The main DABE cost-aware point is EXP-092 at 20.06207 observed bits/token. EXP-087 is the quality anchor. EXP-088, EXP-089, and EXP-090 test variable token-window alternatives. EXP-077 through EXP-087 provide the adaptive-budget development path.

5. Results

5.1. Matched 20 bits/token Comparator

Figure 1 gives the cleanest test of the mechanism claim. At the same bitrate scale, adaptive sparse repair is far more accurate than a fixed-rate no-lookup learned chunk tokenizer.

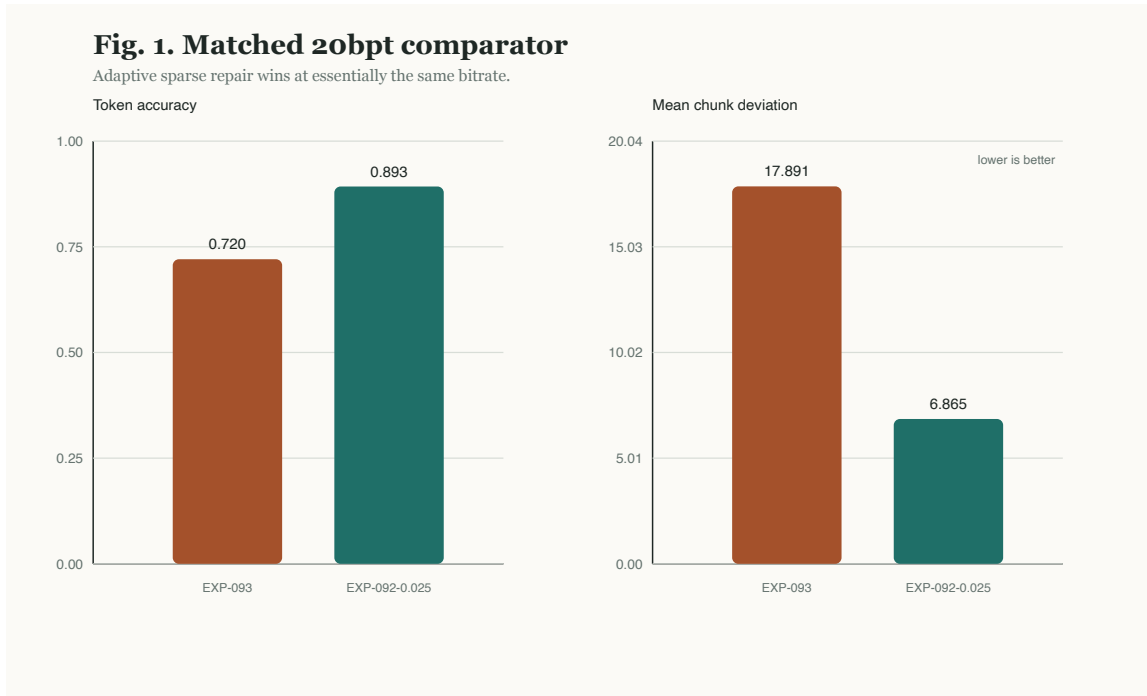


Figure 1: Matched 20 bits/token comparator. Cost-aware sparse repair improves token accuracy and chunk deviation relative to a fixed-rate no-lookup learned compressor at nearly the same bitrate.

Setting	bits/token	Token acc.	Mean dev.	p90 dev.
EXP-093 fixed-rate no lookup	20.00000	0.72045	17.89119	24.14597
EXP-092 cost-aware sparse repair	20.06207	0.89274	6.86454	10.96876

Table 2: Matched 20 bits/token result.

The absolute token-accuracy gain is 0.17229, and mean chunk deviation falls by 11.02665 tokens per 64-token chunk. The fixed-rate baseline is healthy rather than collapsed: its bit density is 0.48766. The gap therefore supports the claim that the gain comes from where lexical precision is spent, not only from how many bits are spent.

5.2. Rate-Distortion Frontier

Figure 2 places the main experiments on a shared rate-distortion plot. Lower chunk deviation at comparable bits/token is better.

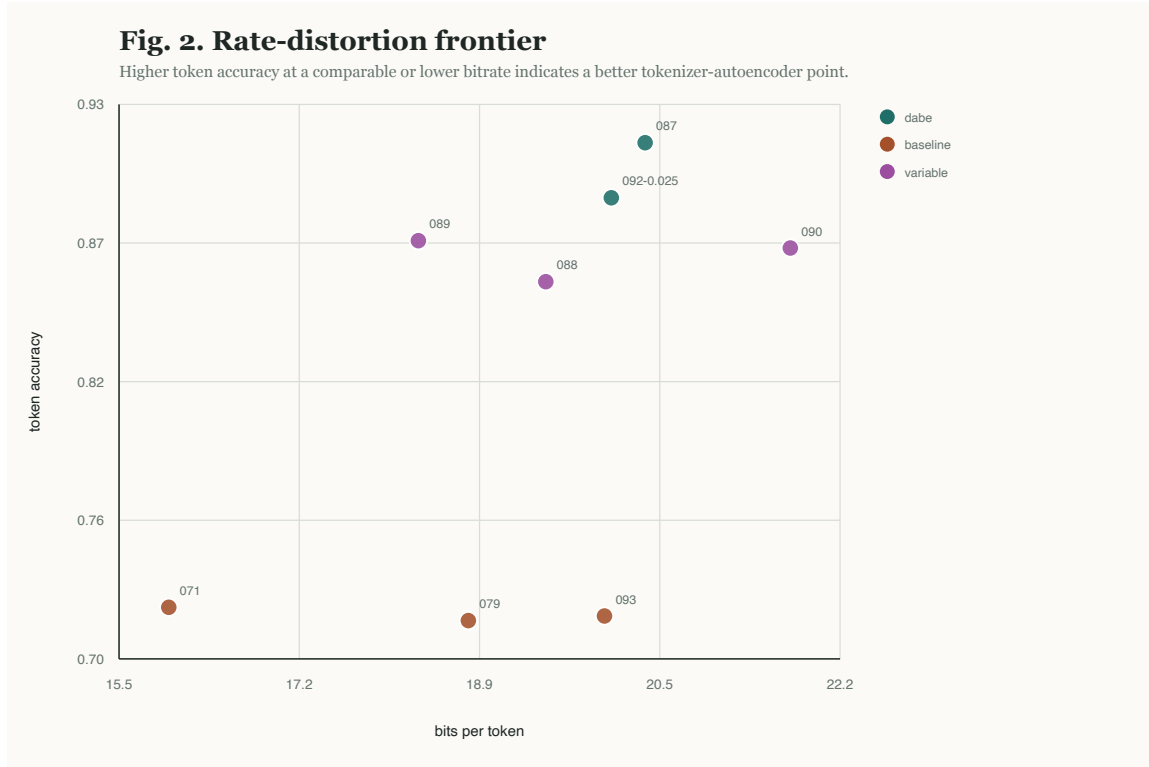


Figure 2: Rate-distortion frontier across fixed-rate baselines, sparse lookup models, cost-aware points, and variable-window ablations. Fixed chunks plus sparse repair form the strongest current frontier.

The best quality anchor is EXP-087, with 0.91547 token accuracy and 5.41007 mean chunk deviation at 20.37236 observed bits/token. The best lower-cost knee is EXP-092 with slot cost 0.025, which reaches 0.89274 token accuracy and 6.86454 mean chunk deviation at 20.06207 observed bits/token.

5.3. Cost-Knee Replication

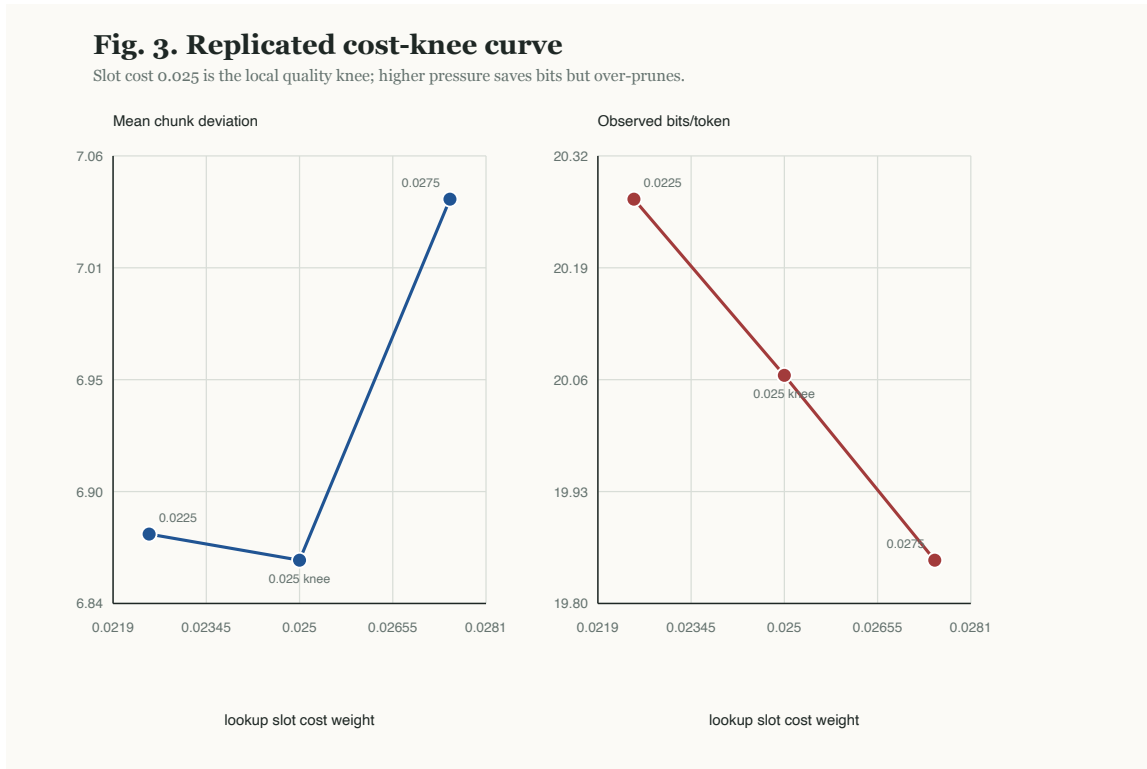


Figure 3: Cost-knee curve for lookup-slot cost. The 0.025 setting is the replicated local knee: lower cost spends more without improving quality, while higher cost over-prunes repair.

Slot cost	bits/token	Active K	Token acc.	Mean dev.
0.0225	20.26545	8.10659	0.89255	6.87713
0.0250	20.06207	6.63953	0.89274	6.86454
0.0275	19.84857	4.54922	0.89002	7.03849

Table 3: Lookup-slot cost sweep around the local knee.

The cost term is useful but not sufficient by itself. It controls active slot usage, but the quality anchor remains EXP-087. This suggests that future gains should improve the router and lookup content at fixed active K , rather than simply increasing cost pressure.

5.4. Adaptive-Budget Ablation

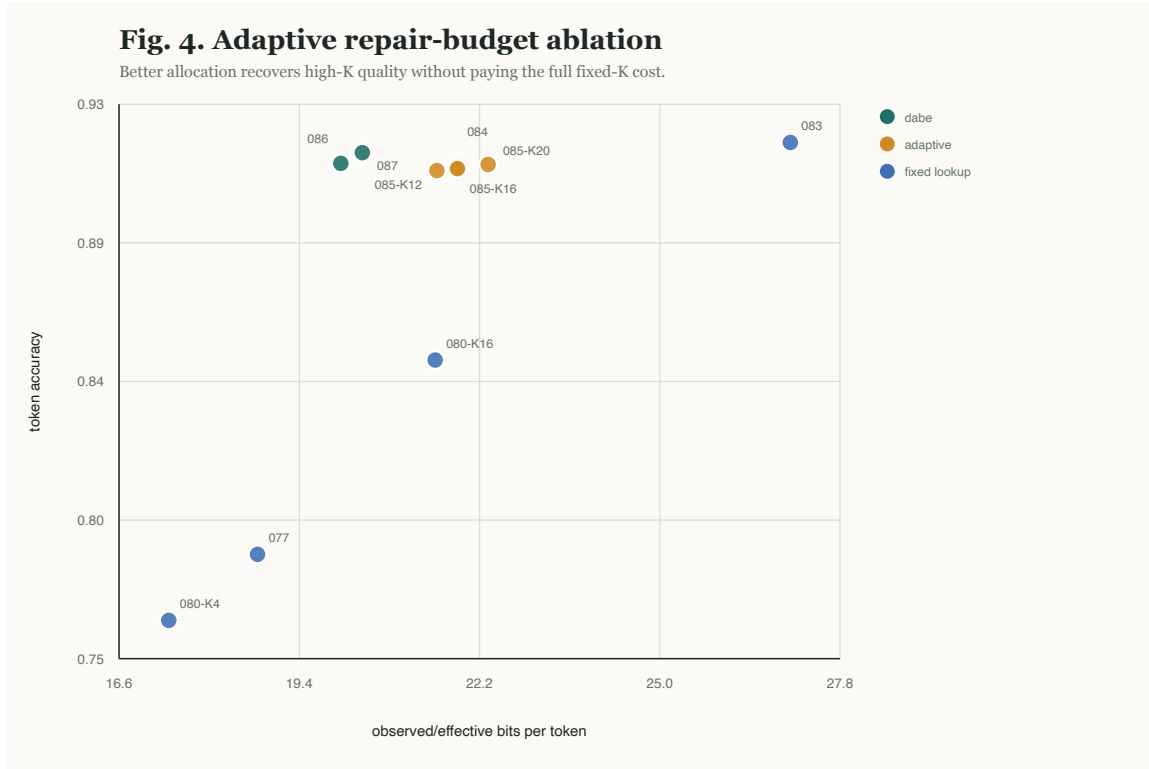


Figure 4: Adaptive-budget ablation. Quality improves as the architecture learns to allocate repair slots more selectively, not merely as raw maximum K increases.

The ablation path starts with fixed sparse lookup, moves to ranked halting, then adds gist and residual-router supervision. Fixed $K = 32$ obtains strong quality at high cost. Ranked halting and gist-residual routing recover much of that quality near the 20–22 bits/token range, motivating the final fixed-chunk plus adaptive sparse repair architecture.

5.5. Variable-Window Ablation

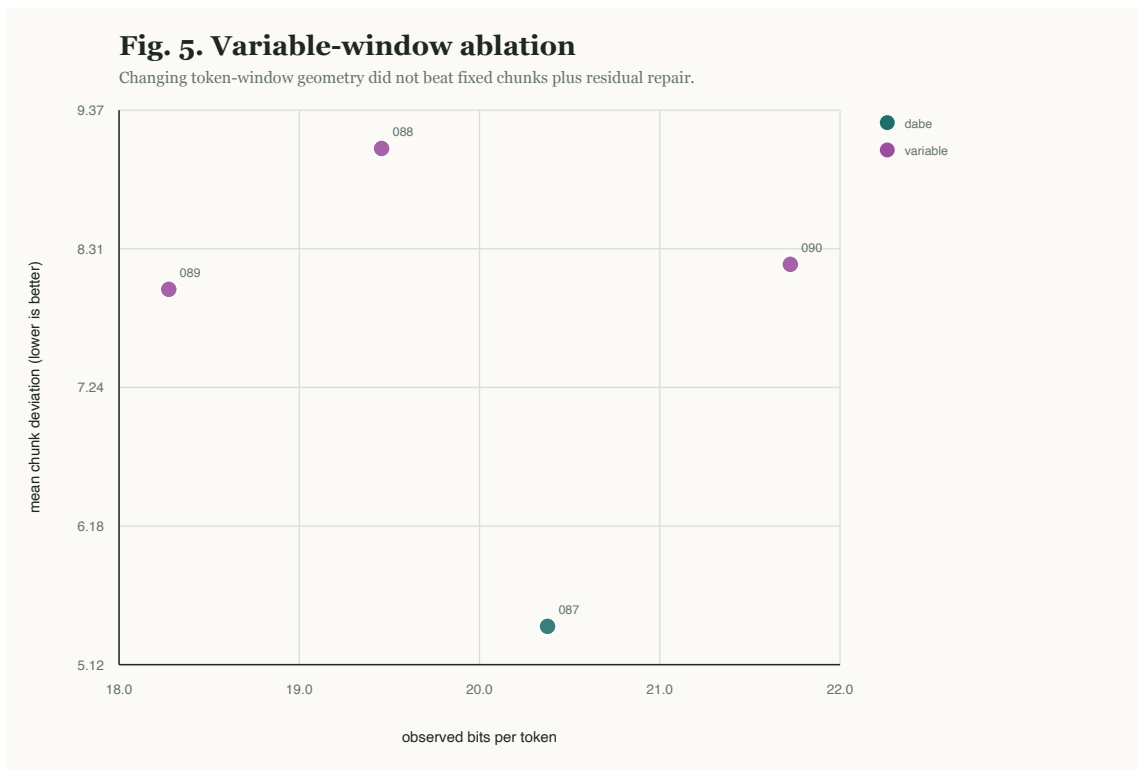


Figure 5: Variable-window ablation. Sliding token-window geometry underperforms the fixed-chunk sparse-repair anchor.

Setting	bits/token	Token acc.	Mean dev.
EXP-088 quantile variable windows	19.46092	0.85820	9.07550
EXP-089 action-value variable windows	18.29175	0.87508	7.99482
EXP-090 hard straight-through windows	21.70567	0.87209	8.18653
EXP-087 fixed chunk + residual lookup	20.37236	0.91547	5.41007

Table 4: Variable-window ablations compared with the fixed-chunk quality anchor.

This ablation narrows the paper’s mechanism claim. Adaptive granularity is useful, but in our experiments it belongs in the repair budget rather than in the primary token-window geometry.

5.6. Active K and Quality

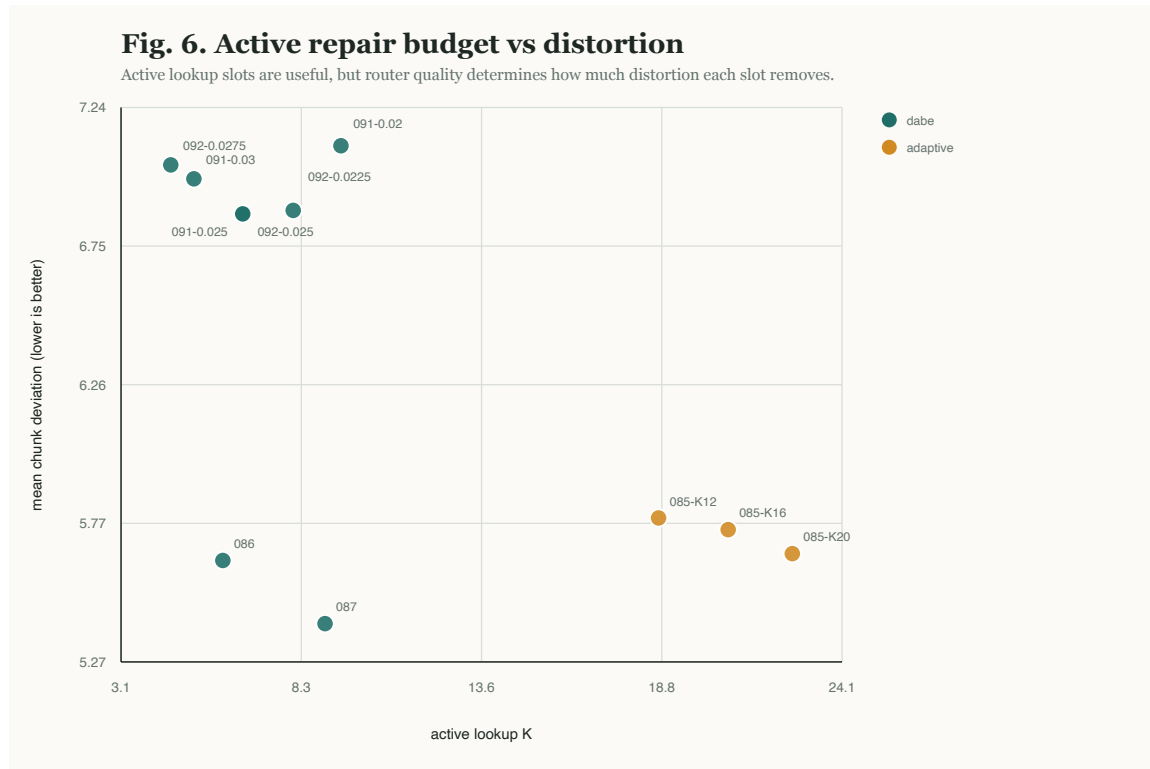


Figure 6: Active repair budget versus distortion. The best points are not simply the largest active- K points; selective allocation matters.

Figure 6 supports the mechanism interpretation. More active slots often help, but the strongest points are produced by architectures that spend slots selectively. This is the empirical counterpart of Equation 15: the side budget must buy useful lexical repairs rather than simply activate more capacity.

5.7. Sparse Repair Trace

Fig. 7. Sparse repair trace from EXP-094

The gist stream misses local lexical details; sparse repair restores selected tokens in the final decode.

Source: EXP-087 checkpoint diagnostic. Gist acc 0.625 -> final acc 0.984; 23 corrected positions.

pos	target	gist pred	final pred	slot p	outcome	why it matters
0	like	really	like	0.51	● corrected	local syntax
5	just	never	just	0.52	● corrected	local syntax
15	covered	are	covered	0.62	● corrected	scene state
17	sun	hole	sun	0.69	● corrected	scene state
21	sky	new	sky	0.50	● corrected	scene state
22	darker	up	darker	0.54	● corrected	scene state
27	What	Thank	What	0.57	● corrected	entity/dialogue detail
30	cloud	one	cloud	0.51	● corrected	entity/dialogue detail
53	strange	new	strange	0.65	● corrected	lexical repair
54	cloud	car	cloud	0.55	● corrected	entity/dialogue detail
60	monster	bad	monster	0.56	● corrected	entity/dialogue detail
61	cloud	dragon	cloud	0.50	● corrected	entity/dialogue detail
63	Anna	And	Anna	0.50	● corrected	entity/dialogue detail

Target: like nothing. It was just a big, white, fluffy cloud. It covered the sun and made the sky darker. "What is that cloud, Anna?" Ben asked, feeling sca...

Figure 7: Sparse repair trace from EXP-094. The gist stream misses scene and dialogue details; the sparse repair path restores selected tokens in the final decode.

Uncorrected gist decode. really toys. It was never a big, Sue's one. It are the hole and saw the new up. "Thank is little one, pictures?" Ben asked, feeling scared. "I are't know, , it is a new car. They it like a bad dragon!" And

Corrected sparse-repair decode. like nothing. It was just a big, white, cloud cloud. It covered the sun and made the sky darker. "What is that cloud, Anna?" Ben asked, feeling scared. "I don't know, Ben, it is a strange cloud. Maybe it is a monster cloud!" Anna

Target reference. like nothing. It was just a big, white, fluffy cloud. It covered the sun and made the sky darker. "What is that cloud, Anna?" Ben asked, feeling scared. "I don't know, Ben, it is a strange cloud. Maybe it is a monster cloud!" Anna

Figure 7 makes the repair mechanism observable. In this diagnostic chunk, the gist stream reconstructs only 62.5% of tokens, confusing scene words such as "sun", "sky", and "cloud" with plausible but wrong alternatives. The repaired decode reaches 98.4% token accuracy on the same chunk, with 23 positions corrected. The trace is the intended interpretation of sparse repair as an inspectable budget, not an opaque black box.

6. Analysis

Why sparse repair works. The gist path turns the chunk code into a smooth whole-span prior. This is useful for predictable text, but exact token reconstruction is brittle around names, punctuation, dialogue markers, and other locally high-information regions. Sparse repair gives the model an explicit way to allocate lexical precision to these holes. Equation 17 makes the consequence measurable: the target is not vague semantic similarity, but fewer wrong tokens per chunk.

Why the fixed-rate comparator matters. EXP-093 is the rate-matched comparator that isolates the mechanism. It spends the same budget scale inside a learned chunk autoencoder, but without the sparse lexical repair path. Its much higher chunk deviation shows that uniform capacity is a weak substitute for adaptive repair.

Code-like text. We added a deterministic Python-code diagnostic corpus for future checkpoint probes. Code stresses different reconstruction behavior than TinyStories: identifiers, indentation, brackets, operators, string delimiters, and numeric literals are sparse but semantically sharp. The same repair trace machinery can test whether lookup slots move from names and narrative content words toward identifiers and syntax-critical punctuation. We treat this as a diagnostic extension rather than a paper claim until the final checkpoint is evaluated on the code corpus.

What remains open. The cost-aware knee shows a path toward better compression, but it also shows that cost pressure alone diminishes on either side of 0.025. The next architectural question is how to improve lookup correctness and router calibration at a fixed active K .

7. Limitations

This study is intentionally framed as tokenizer-autoencoder rate-distortion rather than full downstream language-model pretraining. That choice isolates the tokenization mechanism: the experiments measure how chunk codes, sparse repair, and routing policies affect reconstruction before adding the confounds of language-model scale, optimizer schedules, and downstream task selection. The matched 20 bits/token comparison is therefore the central evidence unit for the paper.

The main experiments use TinyStories as a controlled text domain with exact GPT-2-token reconstruction metrics. This setting makes it possible to compare architectural variants under the same token basis, chunk length, training budget, and validation protocol. Domain generalization remains a separate evaluation target; this setting establishes a clean mechanism result that should next be tested on broader corpora, code, multilingual data, and full LM training.

The run plan prioritizes focused, objective comparisons over broad exploratory sweeps. The experiment set includes a matched fixed-rate comparator, a replicated cost-knee neighborhood around EXP-092, a quality anchor in EXP-087, and direct variable-window ablations. Additional seeds and larger-scale runs would support confidence intervals and scaling laws, but they are not required to interpret the main matched-rate separation.

The current models are rate-distortion tokenizers, not lossless compressors. Exact full-chunk reconstruction remains low, while token accuracy, exact local blocks, and mean chunk deviation improve substantially with sparse repair. This is the correct operating point for the paper’s claim: DABE demonstrates that adaptive repair allocation can sharply reduce reconstruction distortion at a matched observed bitrate.

8. Conclusion

DABE studies tokenization as a learned rate-distortion problem over fixed text chunks. The strongest result is not obtained by changing token-window geometry, but by keeping a stable 64-token chunk and adapting a sparse lexical repair budget over it.

The matched comparison between EXP-093 and EXP-092 is the central result: a healthy fixed-rate no-lookup learned tokenizer at 20.0 bits/token reaches 0.72045 token accuracy and 17.89119 mean chunk deviation, while cost-aware DABE reaches 0.89274 token accuracy and 6.86454 mean chunk deviation at 20.06207 observed bits/token. This shows that total bit budget alone does not explain the gain.

The broader ablation story is consistent. Fixed sparse lookup improves over no-lookup codes; ranked halting recovers much of fixed $K = 32$ quality at lower observed bitrate; the gist-residual architecture improves repair allocation further; and variable token-window geometry fails to beat fixed chunks plus repair. The architectural implication is clear: for learned tokenizers, adapt the repair budget before adapting the token window.

Bibliography

- [1] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- [2] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.
- [3] M. Schuster and K. Nakajima, “Japanese and Korean voice search,” in *2012 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2012.
- [4] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural Discrete Representation Learning,” in *Advances in Neural Information Processing Systems*, 2017.
- [5] L. Kaiser *et al.*, “Fast Decoding in Sequence Models using Discrete Latent Variables,” in *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [6] A. Graves, “Adaptive Computation Time for Recurrent Neural Networks,” *arXiv preprint arXiv:1603.08983*, 2016.
- [7] Y. Bengio, N. Leonard, and A. Courville, “Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [8] J. Gu, Z. Lu, H. Li, and V. O. K. Li, “Incorporating Copying Mechanism in Sequence-to-Sequence Learning,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- [9] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020.